

README

Part 3: 训练诊断与 BatchNorm — 让训练稳定起来

> 模型训练不收敛？Loss 下不去？这一课教你给网络做体检！

章节导航

序号	章节	内容
01	[训练诊断：你的模型生病了](01_diagnosis.md)	初始 Loss 异常、tanh 饱和、Kaiming 初始化
02	[BatchNorm：深度学习的维生素](02_batchnorm.md)	BN 原理、从零实现、常见陷阱
03	[深层网络与诊断工具](03_deep_network.md)	多层网络搭建、4 种诊断工具、更新/数据比率

学习路线图

Part 2 (MLP)

|
| "MLP 训练起来了，但总觉得哪里不对劲..."



	Part 3: 训练诊断与 BatchNorm	
	① 给模型做体检 — 找到病因	———> 01_diagnosis.md
	② 开处方 — BatchNorm	———> 02_batchnorm.md
	③ 深层网络 + 诊断工具箱	———> 03_deep_network.md

|
| "网络稳定了，想深入理解反向传播..."



Part 4 (手动反向传播)

学完这一部分你能...

- 诊断初始 Loss 异常 — 为什么第一轮 loss 就该 ≈ 3.29
- 理解 tanh 饱和（梯度消失）的原因和修复方法
- 掌握 Kaiming 初始化 — 让每一层的激活值保持合理范围
- 从零实现 BatchNorm，理解训练/评估两种模式
- 搭建深层网络，用 4 种诊断工具监控训练健康度

课后作业

完成教程后，去这里做练习：

[Assignment 3](../../assignments/assignment_3/)

相关资源

- Andrej Karpathy 原视频：[Building makemore Part 3: Activations & Gradients](https://www.youtube.com/watch?v=P6sfmUTpUmc)
- Ioffe & Szegedy 2015 论文：[Batch Normalization: Accelerating Deep Network Training](https://arxiv.org/abs/1502.03167)
- He et al. 2015 论文：[Delving Deep into Rectifiers (Kaiming Init)](https://arxiv.org/abs/1502.01852)

[← 上一章：Part 2 MLP](../../Part2_mlp/tutorial/README.md) | [下一章：Part 4 Backpropagation →](../../Part4_backprop/tutorial/README.md)

01_diagnosis

01 — 训练诊断：你的模型生病了

前置知识：MLP 的"亚健康"状态

在 [Part 2](../../Part2_mlp/) 里，我们搭了一个 MLP —— Embedding → 隐藏层 → 输出层，训练后 loss 能降到 ~2.17。

看起来还行？但 Andrej Karpathy 说：如果初始化做对了，你能白嫖一大截性能。

这一课，我们就来当"神经网络医生"，学会诊断两个最常见的初始化问题。

症状 1：初始 Loss 过高

期望 vs 现实

在训练开始之前（随机初始化），模型对 27 个字符一无所知 —— 每个字符的概率应该是均匀分布 $1/27$ 。

对应的交叉熵 loss：

```
expected_loss = -ln(1/27) = ln(27) ≈ 3.29
```

△ 但如果你跑一下没修过的网络，初始 loss 可能是 20 甚至 30+！

这意味着模型对某些字符"过于自信"（给了很高的概率），结果猜错了被打脸。损失函数给了极大的惩罚。

> 完整诊断代码见 [\[../scripts/01_diagnose_initial_loss.py\]](#)(../scripts/01_diagnose_initial_loss.py)

怎么修？

问题出在输出层。初始化时最后一层的 logits 太大了，导致 softmax 输出非常尖锐。

修复方法很简单——把输出层的权重缩小：

```
`python
```

修复前

```
W2 = torch.randn((n_hidden, vocab_size))
```

修复后：缩小输出层权重

```
W2 = torch.randn((n_hidden, vocab_size)) * 0.01
```

```
b2 = torch.randn(vocab_size) * 0
```

初始 Loss 对比：

修复前:  20.0+

修复后:  3.29

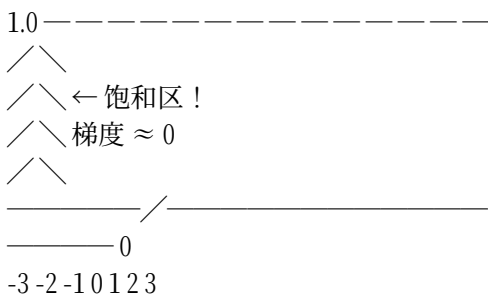
差距：训练初期浪费的步数 = 白跑的 epoch！

关键洞察：初始 loss = 3.29 意味着模型说"我不知道"，这是最诚实的起点。

症状 2：tanh 饱和（梯度消失）

什么是 tanh 饱和？

tanh 函数长这样：



当 tanh 的输入绝对值很大时（比如 > 3 ），输出接近 1 或 -1，梯度几乎为 0。

这意味着：这个神经元在反向传播时"罢工"了——梯度传不过去！

> 诊断代码见 [\[../scripts/02_diagnose_tanh_saturation.py\]](#)(../scripts/02_diagnose_tanh_saturation.py)

> 可视化见下面的 tanh 饱和度直方图

怎么诊断？

在训练初始化后，统计隐藏层 $h = \tanh(h_{\text{preact}})$ 的值：

```
`python
h = torch.tanh(hpreact) # 隐藏层输出
```

统计饱和程度

```
saturated = (h.abs() > 0.97).float().mean()
print(f'饱和比例: {saturated * 100:.1f}%') # 希望这个数很小
```

```
`python
```

可视化：tanh 层激活值分布直方图 → 生成
../images/cell015_output02.png

```
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 4))
legends = []
for i, layer in enumerate(layers[:-1]): # 排除输出层
    if isinstance(layer, Tanh):
        t = layer.out
        print(f'layer {i} ({layer.__class__.__name__}): '
              f'mean {t.mean():+.2f}, std {t.std():.2f}, '
              f'saturated: {(t.abs() > 0.97).float().mean()*100:.2f}%')
        hy, hx = torch.histogram(t, density=True)
        plt.plot(hx[:-1].detach(), hy.detach())
        legends.append(f'layer {i} ({layer.__class__.__name__})')
plt.legend(legends)
plt.title('Activation Distribution')
plt.savefig('../images/cell015_output02.png', dpi=150, bbox_inches='tight')
plt.show()
```

> 完整脚本见 [\[scripts/06_diagnostic_tools.py\]](#)(../scripts/06_diagnostic_tools.py)

![tanh 饱和度直方图](../images/cell015_output02.png)

如果大部分值都挤在 -1 和 1 附近，说明 tanh 饱和严重。

为什么饱和 = 梯度消失？

反向传播时 tanh 的梯度是 $1 - t^2$ （ t 是 tanh 输出）：

tanh 饱和时的梯度链：

Loss → ... → $\tanh_grad(1 - t^2)$ → ... → 输入

当 $t \approx 1$ 或 $t \approx -1$ 时：

$1 - t^2 \approx 1 - 1 = 0 \leftarrow$ 梯度被"掐断"了！

多层叠加后：

梯度 $\times 0 \times 0 \times 0 \times \dots \approx 0 \leftarrow$ 完全消失

,

解药：Kaiming 初始化

核心思想

我们希望每一层的输出方差 $\approx$ 输入方差，这样信号在多层网络中不会爆炸也不会消失。

He et al. (2015) 提出了 Kaiming 初始化：

python

标准初始化（不好）

`W = torch.randn(fan_in, fan_out)`

Kaiming 初始化（好）

`W = torch.randn(fan_in, fan_out) (gain / fan_in * 0.5)`

,

其中 `gain` 取决于激活函数：

激活函数	gain 值	说明
<code>**ReLU**</code>	$\sqrt{2} \approx 1.41$	最常用
<code>**tanh**</code>	$5/3 \approx 1.67$	我们用的
Linear（无激活）	1.0	线性层

> 完整代码见 `[../scripts/03_kaiming_init.py](../scripts/03_kaiming_init.py)`

为什么是 `fan_in^0.5`？

直觉理解：如果输入有 `fan_in` 个元素，做点积后结果的方差会放大 `fan_in` 倍。除以 $\sqrt{\text{fan_in}}$ 就是抵消这个放大。

,

未初始化时，每层方差的变化：

Layer 1: std=1.0 $\rightarrow$ Layer 2: std=5.3 $\rightarrow$ Layer 3: std=28.0 $\rightarrow$ 爆炸！

Kaiming 初始化后：

Layer 1: std=1.0 $\rightarrow$ Layer 2: std=1.0 $\rightarrow$ Layer 3: std=1.0 $\rightarrow$ 稳定！

,

对我们的 MLP 意味着什么？

```
`python
```

隐藏层权重用 Kaiming 初始化

```
W1 = torch.randn((n_embd block_size, n_hidden)) (5/3) / ((n_embd block_size) * 0.5)
```

输出层权重缩小（避免初始 loss 过高）

```
W2 = torch.randn((n_hidden, vocab_size)) * 0.01  
b2 = torch.zeros(vocab_size)
```

两步修复后，我们的 MLP：

- 初始 loss 从 20+ $\rightarrow$ 3.29
- tanh 饱和比例大幅下降
- 训练更稳定，收敛更快

课后练习

练习 1：验证初始 Loss

> 不看代码，自己写一段程序：随机初始化一个 (200, 27) 的权重矩阵，计算均匀分布下的交叉熵 loss。验证是不是 ≈ 3.29 。

<details>

<summary> 提示</summary>

用 `F.cross_entropy(logits, targets)` 计算。初始化 logits 时权重乘以 0.01，看 loss 是否接近 $-\ln(1/27)$ 。

</details>

练习 2：tanh 饱和实验

> 修改 Kaiming 初始化中的 gain 值：分别用 gain=0.1、1.0、5/3、3.0 初始化权重，统计 tanh 饱和比例 ($|h| > 0.97$)，画出对比图。

<details>

<summary> 提示</summary>

对每种 gain 值，运行一次前向传播，计算 `(h.abs() > 0.97).float().mean()`。观察 gain 太小或太大时发生什么。

</details>

练习 3：为什么是 5/3？

> tanh 的 gain = 5/3。你能用直觉解释为什么 tanh 的 gain 比 ReLU 的 $\sqrt{2}$ 小吗？（提示：tanh 会把输出“压缩”到 $[-1, 1]$ ，而 ReLU 只在负半轴压缩为 0）

下一步

诊断和初始化做好了，但还有一个大招没出 —— BatchNorm !

[02 — BatchNorm：深度学习的维生素](02_batchnorm.md)

02_batchnorm

02 — BatchNorm：深度学习的维生素

为什么需要 BatchNorm ?

上一课我们学了一个"朴素"的方案：用 Kaiming 初始化让每层的输出保持合理范围。

但这个方案有个问题 ——

它只在初始化时管用。训练开始后，权重不断更新，各层的激活分布还是会逐渐跑偏。

BatchNorm 的核心想法：别只在初始化时修一次，每一步都自动矫正！

就像维生素一样，每天吃一粒，保持身体的"均值和方差"在正常范围内

BN 原理

第一步：标准化 (Normalize)

对每个 mini-batch 的激活值，减均值、除标准差：

```
`python
```

hpreact 形状: (batch_size, n_hidden)

沿 batch 维度 (dim=0) 计算均值和方差

```
bnmean = hpreact.mean(0, keepdim=True) # (1, n_hidden)
```

```
bnstd = hpreact.std(0, keepdim=True) # (1, n_hidden)
```

标准化

```
hpreact_norm = (hpreact - bnmean) / bnstd # 均值 0, 方差 1
```

标准化后，每个神经元的激活值大致在 $[-2, 2]$ 范围内 —— tanh 不会饱和了。

第二步：可学习参数 γ 和 β

光标准化还不够——如果强行把所有值压成均值 0、方差 1，网络的表达能力就受限了。

所以 BN 引入了两个可学习参数：

```
`python
```

γ (gamma/bngain): 缩放参数

β (beta/bnbias): 偏移参数

```
hpreact = bngain * hpreact_norm + bnbias
```

标准化后的分布：

```
-2 -1 0 1 2 ← 均值 0, 方差 1  
| | | | |
```

经过 γ 缩放和 β 偏移后：

```
-1 0 1 2 3 4 5 ←  $\gamma=2, \beta=3$  时  
| | | | | | |
```

网络可以自己学需要什么样的分布。初始时 $\gamma=1$ 、 $\beta=0$ （即不改变标准化结果），训练过程中慢慢调整。

第三步：Running Mean/Var（推理时用）

训练时我们用 mini-batch 的均值和方差。但推理时可能只有一个样本（batch_size=1），怎么算均值方差？

解决方案：训练时维护一个指数移动平均（EMA）的 running mean 和 running var：

```
`python
```

```
bnmean_running = torch.zeros((1, n_hidden))  
bnstd_running = torch.ones((1, n_hidden))
```

训练循环中，每次更新 running stats

这里用 0.999/0.001 的系数（等价于 momentum=0.001），更新较慢但更稳定

```
with torch.no_grad():  
    bnmean_running = 0.999 bnmean_running + 0.001 bnmean  
    bnstd_running = 0.999 bnstd_running + 0.001 bnstd
```

> momentum 参数说明：PyTorch 的 `nn.BatchNorm1d` 默认 `momentum=0.1`（即 `0.9 running + 0.1 batch`），更新较快。上面的内联代码用 `0.001`（即 `0.999 running + 0.001 batch`），更新较慢但更平滑。两种都可以，区别在于 running stats 的收敛速度。下面的 `BatchNorm1d` 类使用 `momentum=0.1`（PyTorch 默认值）。

训练模式：用当前 batch 的 mean/std（精确）

评估模式：用 running mean/std（全局近似）

两者切换：

model.train() → 用 batch stats

model.eval() → 用 running stats

从零实现

> 完整代码见 [\[./scripts/04_batchnorm_implementation.py\]](#)([../scripts/04_batchnorm_implementation.py](#))

BN 在网络中的位置

Embedding → [Linear → BatchNorm → Tanh] × N → Linear → 输出

↑

每个隐藏层后面

都跟一个 BN

完整实现

```
python
class BatchNorm1d:
    def __init__(self, dim, eps=1e-5, momentum=0.1):
        self.eps = eps # 防止除零的小数
        self.momentum = momentum # EMA 的更新速率
        self.training = True # 训练 or 评估模式
```

可学习参数（通过反向传播更新）

```
self.gamma = torch.ones(dim) # 缩放
```

```
self.beta = torch.zeros(dim) # 偏移
```

缓冲区（通过 EMA 更新，不参与反向传播）

```
self.running_mean = torch.zeros(dim)
```

```
self.running_var = torch.ones(dim)
```

```
def __call__(self, x):
```

```
    if self.training:
```

```
        xmean = x.mean(0, keepdim=True)
```

```
        xvar = x.var(0, keepdim=True)
```

```
    else:
```

```
        xmean = self.running_mean
```

```
        xvar = self.running_var
```

标准化

```
xhat = (x - xmean) / torch.sqrt(xvar + self.eps)
self.out = self.gamma * xhat + self.beta
```

训练时更新 running stats

```
if self.training:
    with torch.no_grad():
        self.running_mean = (1 - self.momentum) self.running_mean + self.momentum xmean
        self.running_var = (1 - self.momentum) self.running_var + self.momentum xvar

    return self.out

def parameters(self):
    return [self.gamma, self.beta]
```

在 MLP 中使用

```
`python
C = torch.randn((vocab_size, n_embd), generator=g)
W1 = torch.randn((n_embd block_size, n_hidden), generator=g) (5/3)/((n_embd block_size)*0.5)
W2 = torch.randn((n_hidden, vocab_size), generator=g) * 0.01
b2 = torch.randn(vocab_size, generator=g) * 0
```

BN 参数

```
bn gain = torch.ones((1, n_hidden))
bn bias = torch.zeros((1, n_hidden))
bn mean_running = torch.zeros((1, n_hidden))
bn std_running = torch.ones((1, n_hidden))
```

前向传播

```
emb = C[Xb]
embcat = emb.view(emb.shape[0], -1)
hpreact = embcat @ W1
```

---- BatchNorm 层 ----

```
bn mean_i = hpreact.mean(0, keepdim=True)
bn std_i = hpreact.std(0, keepdim=True)
hpreact = bn gain * (hpreact - bn mean_i) / bn std_i + bn bias

with torch.no_grad():
    bn mean_running = 0.999 bn mean_running + 0.001 bn mean_i
    bn std_running = 0.999 bn std_running + 0.001 bn std_i
```

```
-----

h = torch.tanh(hpreact)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Yb)
```

推理时

```
`python
@torch.no_grad()
def split_loss(split):
    x, y = {'train': (Xtr, Ytr), 'val': (Xdev, Ydev)}[split]
    emb = C[x]
    embcat = emb.view(emb.shape[0], -1)
    hpreact = embcat @ W1
```

用 running stats 代替 batch stats

```
hpreact = bngain * (hpreact - bnmean_running) / bnstd_running + bnbias
h = torch.tanh(hpreact)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, y)
print(split, loss.item())
`
```

⚠ BN 的坑

坑 1：忘记切 training/eval 模式

```
`python
    训练完直接评估，忘了切模式
```

`model.eval()` # ← 必须调这个！

或者手动设置

```
for layer in layers:
    if isinstance(layer, BatchNorm1d):
        layer.training = False
`
```

如果你不切换，推理时 BN 还在用 batch 的均值方差 —— `batch_size=1` 时结果会很离谱。

坑 2：`batch_size=1` 时方差为 NaN

```
`python
batch_size=1 时，方差 = 0（只有一个样本！）
```

$\text{std} = \sqrt{0} = 0 \rightarrow \text{除以 } 0 \rightarrow \text{NaN}$

解决方案：

1. 加 eps（我们设了 $\text{eps}=1\text{e-}5$ ）
2. 推理时用 running stats（而不是 batch stats）

坑 3：Running Mean 没收敛

```
python
```

momentum=0.1 时，大约需要 30-50 步让 running mean 稳定

如果训练步数太少（比如只跑了 100 步），running mean 可能不准

解决方案：

1. 跑够步数（我们跑了 200000 步，没问题）
2. 训练结束后，用整个训练集重新校准一次

```
with torch.no_grad():  
    emb = C[Xtr]  
    hpreact = emb.view(emb.shape[0], -1) @ W1  
    bnmean = hpreact.mean(0, keepdim=True) # 精确的全局均值  
    bnstd = hpreact.std(0, keepdim=True) # 精确的全局标准差
```

课后练习

练习 1：去掉 BN 会怎样？

> 把网络中的 BatchNorm 层去掉，只保留 Linear + Tanh。观察训练 loss 曲线有什么变化。然后加回 BN，对比 loss 收敛速度。

<details>

<summary> 提示</summary>

去掉 BN 后，深层网络的 loss 曲线通常会：

- 初始 loss 更高
- 训练过程更不稳定（抖动大）
- 最终 loss 也可能更高

</details>

练习 2：自己实现 EMA

> 不用 PyTorch 的 `running_mean`，自己维护一个 Python 列表记录每一步的 batch mean。训练结束后，画出 batch mean 随训练步数的变化曲线。观察大约多少步后 running mean 趋于稳定。

<details>

<summary> 提示</summary>

```
`python
mean_history = []
for i in range(max_steps):
... forward pass ...

mean_history.append(bnmean.mean().item())
```

画出 mean_history

```
plt.plot(mean_history)
plt.xlabel('step')
plt.ylabel('batch mean')
```

</details>

练习 3：Gamma 和 Beta 的作用

> 初始化时让 $\gamma=0.5$ （而不是 1.0），观察 tanh 饱和度有什么变化。再让 $\beta=1.0$ （而不是 0），观察 tanh 输出的分布有什么偏移。

下一步

BN 学完了，现在我们有了一个稳定的“积木”。接下来用这些积木搭建更深的网络，并学习一套完整的诊断工具箱！

[03 — 深层网络与诊断工具](03_deep_network.md)

03_deep_network

03 — 深层网络与诊断工具

构建深层网络

前面我们一直在用一个两层 MLP（一个隐藏层）。现在把学到的知识组合起来，搭一个更深的网络！

> 完整代码见 ../scripts/05_deep_network.py

用列表管理多层

关键技巧：把每一层放进一个列表里，前向传播时循环调用：

```
`python
class Linear:
def __init__(self, fan_in, fan_out, bias=True):
```

Kaiming 初始化

```
self.weight = torch.randn((fan_in, fan_out), generator=g) (5/3) / fan_in*0.5
self.bias = torch.zeros(fan_out) if bias else None
```

```
def __call__(self, x):
self.out = x @ self.weight
if self.bias is not None:
self.out += self.bias
return self.out

def parameters(self):
return [self.weight] + ([self.bias] if self.bias is not None else [])
```

```
class Tanh:
def __call__(self, x):
self.out = torch.tanh(x)
return self.out

def parameters(self):
return []
```

6 层网络结构

```
`python
n_embd = 10
n_hidden = 100

C = torch.randn((vocab_size, n_embd), generator=g)

layers = [
Linear(n_embd * block_size, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
Linear(n_hidden, n_hidden, bias=False), BatchNorm1d(n_hidden), Tanh(),
Linear(n_hidden, vocab_size, bias=False), BatchNorm1d(vocab_size),
]
```

网络结构（5 个隐藏层 + 1 个输出层）：

Input → Embedding



```
| Linear → BatchNorm → Tanh ← 隐藏层 1 |  
| Linear → BatchNorm → Tanh ← 隐藏层 2 |  
| Linear → BatchNorm → Tanh ← 隐藏层 3 |  
| Linear → BatchNorm → Tanh ← 隐藏层 4 |  
| Linear → BatchNorm → Tanh ← 隐藏层 5 |  
| Linear → BatchNorm ← 输出层 |
```



CrossEntropy Loss

注意每层后面都跟了 BatchNorm！这是深层网络训练稳定的关键。

> 代码风格说明：上面用类（`Linear`、`Tanh`、`BatchNorm1d`）来展示网络结构，更直观。实际脚本 `[05_deep_network.py](../scripts/05_deep_network.py)` 中使用了更简洁的字典结构（`{'type': 'linear', 'W': W, 'b': b}`），效果完全相同。两种写法都是正确的，类写法更易读，字典写法更紧凑。

初始化技巧

```
`python  
with torch.no_grad():
```

最后一层：降低置信度

```
layers[-1].gamma *= 0.1
```

其他层：Kaiming gain 已经在 `Linear.__init__` 里处理了

4 种诊断工具

> 诊断代码见 `[../scripts/06_diagnostic_tools.py](../scripts/06_diagnostic_tools.py)`

工具 1：激活值分布（Activation Distribution）

```
`python
```

可视化：各层激活值直方图 → 生成
`../images/cell015_output02.png`

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(20, 4))
```

```

legends = []
for i, layer in enumerate(layers[:-1]):
    if isinstance(layer, Tanh):
        t = layer.out
        hy, hx = torch.histogram(t, density=True)
        plt.plot(hx[:-1].detach(), hy.detach())
        legends.append(f'layer {i} ({layer.__class__.__name__})')
plt.legend(legends)
plt.title('Activation Distribution')
plt.savefig('../images/cell015_output02.png', dpi=150, bbox_inches='tight')
plt.show()

```

![激活值分布](../images/cell015_output02.png)

统计每个 Tanh 层输出的均值、标准差和饱和比例：

```

python
for i, layer in enumerate(layers):
    if isinstance(layer, Tanh):
        t = layer.out
        print(f"layer {i} ({layer.__class__.__name__}): "
              f"mean {t.mean():+.2f}, std {t.std():.2f}, "
              f"saturated: {(t.abs() > 0.97).float().mean()*100:.2f}%")

```

健康输出示例：

```

layer 2 (Tanh): mean -0.00, std 0.63, saturated: 2.78%
layer 5 (Tanh): mean +0.00, std 0.64, saturated: 2.56%
layer 8 (Tanh): mean -0.00, std 0.65, saturated: 2.25%
layer 11 (Tanh): mean +0.00, std 0.65, saturated: 1.69%
layer 14 (Tanh): mean +0.00, std 0.65, saturated: 1.88%

```

健康的标准：各层 std 接近、饱和率低（< 5%）、mean 接近 0。

工具 2：梯度分布（Gradient Distribution）

```

python
可视化：各层梯度直方图 → 生成
../images/cell016_output02.png

```

```

import matplotlib.pyplot as plt

plt.figure(figsize=(20, 4))
legends = []
for i, layer in enumerate(layers[:-1]):
    if isinstance(layer, Tanh):
        t = layer.out.grad
        hy, hx = torch.histogram(t, density=True)
        plt.plot(hx[:-1].detach(), hy.detach())
        legends.append(f'layer {i} ({layer.__class__.__name__})')

```



```
plt.legend(legends)
plt.title('Gradient Distribution')
plt.savefig('../images/cell016_output02.png', dpi=150, bbox_inches='tight')
plt.show()
```

![梯度分布](../images/cell016_output02.png)

看反向传播时各层 Tanh 的梯度分布：

```
`python
for i, layer in enumerate(layers):
    if isinstance(layer, Tanh):
        t = layer.out.grad # 梯度 !
        print(f"layer {i}: grad mean {t.mean():+e}, grad std {t.std():e}")
```

```
layer 2: grad mean -0.000000, grad std 2.64e-03
layer 5: grad mean +0.000000, grad std 2.25e-03
layer 8: grad mean -0.000000, grad std 2.05e-03
layer 11: grad mean +0.000000, grad std 1.89e-03
layer 14: grad mean +0.000000, grad std 1.68e-03
```

健康的标准：各层梯度 std 接近，没有某一层梯度突然缩小（梯度消失）或放大（梯度爆炸）。

工具 3：参数梯度/数据比率（Grad:Data Ratio）

```
`python
```

可视化：参数梯度分布 → 生成 ../images/cell017_output01.png

```
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 4))
legends = []
for i, p in enumerate(parameters):
    t = p.grad
    if p.ndim == 2:
        hy, hx = torch.histogram(t, density=True)
        plt.plot(hx[:-1].detach(), hy.detach())
        legends.append(f'{i} {tuple(p.shape)}')
plt.legend(legends)
plt.title('Weights Gradient Distribution')
plt.savefig('../images/cell017_output01.png', dpi=150, bbox_inches='tight')
plt.show()
```

![参数梯度比率](../images/cell017_output01.png)

这是最重要的指标！它告诉我们：参数的梯度有多大，相对于参数本身有多大。

```
`python
for i, p in enumerate(parameters):
    if p.ndim == 2:
```

```

grad_std = p.grad.std()
data_std = p.data.std()
ratio = grad_std / data_std
print(f'weight {str(p.shape):>12s} | "
f"grad:data ratio {ratio:.3e}"')
`
`

weight (27, 10) | grad:data ratio 8.01e-03
weight (30, 100) | grad:data ratio 4.88e-02
weight (100, 100) | grad:data ratio 5.14e-02
...
`

```

含义：如果 ratio 太大，说明梯度比参数大很多 —— 参数更新的步子太大了，训练会不稳定。如果 ratio 太小，学习太慢。

工具 4：更新/数据比率（Update:Data Ratio） 最重要

python
 可视化：更新/数据比率随训练步数变化 → 生成
 ../images/cell018_output00.png

```

import matplotlib.pyplot as plt

plt.figure(figsize=(20, 4))
legends = []
for i, p in enumerate(parameters):
    if p.ndim == 2:
        plt.plot([ud[j][i] for j in range(len(ud))])
        legends.append(f'param {i}')
plt.plot([0, len(ud)], [-3, -3], 'k') # 理想值 ~1e-3
plt.legend(legends)
plt.title('Update-to-Data Ratio (log scale)')
plt.savefig('../images/cell018_output00.png', dpi=150, bbox_inches='tight')
plt.show()
`

```

![更新数据比率](../images/cell018_output00.png)

python
 每步记录

```

ud = []
with torch.no_grad():
    ud.append([((lr * p.grad).std() / p.data.std()).log10().item()
    for p in parameters])

```

画图

```

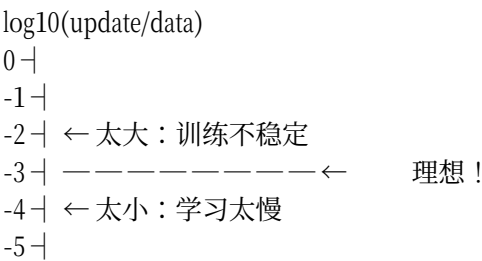
plt.figure(figsize=(20, 4))
for i, p in enumerate(parameters):

```

```
if p.ndim == 2:
plt.plot([ud[j][i] for j in range(len(ud))])
plt.plot([0, len(ud)], [-3, -3], 'k') # 理想参考线
```

Andrej 的经验法则：更新/数据比率的 log10 应该在 -3 左右（即 $\text{lr grad.std} \approx 0.001 \text{ data.std}$ ）。

理想范围：



如果某个参数的曲线远远偏离 -3，说明学习率需要调整，或者初始化有问题。

四种诊断工具总结

工具	看什么	健康标准
激活值分布	前向传播中各层的输出	std 接近、饱和率 < 5%
梯度分布	反向传播中各层的梯度	各层梯度 std 接近
梯度/数据比率	梯度相对参数的大小	各层接近、无极端值
更新/数据比率	实际更新步长相对参数	**log10 $\approx$ -3**

诊断顺序：先看更新/数据比率（最重要），如果不对，再往前追溯梯度分布和激活值分布。

课后作业

[Assignment 3](../..../assignments/assignment_3/)

作业内容提示：

- 修改网络深度和宽度，观察对训练的影响
- 去掉 BatchNorm，用 Kaiming 初始化替代，比较效果
- 调整学习率，用更新/数据比率判断学习率是否合适

下一课预告

Part 3 我们学会了诊断和稳定训练。但所有东西都依赖 PyTorch 的 `autograd` 自动微分 —— 它是怎么工作的？

在 Part 4 里，我们将手动实现反向传播！不靠

`loss.backward()`，自己算每一层的梯度。理解了这个，你才算真正懂了神经网络。

Part 3 总结：

诊断工具箱 治疗方案

初始 Loss 过高	————→	输出层权重缩小
tanh 饱和	————→	Kaiming 初始化
梯度消失/爆炸	————→	BatchNorm
更新比率异常	————→	调学习率

[\[返回目录\]\(README.md\)](#)